FUNDAÇÃO EDUCACIONAL INACIANA PADRE SABÓIA DE MEDEIROS

CIÊNCIA DA COMPUTAÇÃO

ANA BEATRIZ MORETTI TURINI

22.126.056-5

ESDRAS VINÍCIUS CORTEZ QUEIROZ

22.126.048-2

PROJETO DE COMPUTAÇÃO EMBARCADA MP3

DISCIPLINA DE DIGITAL EXPERIENCE ULTIMATE CCP020

SÃO BERNARDO DO CAMPO-SP

2026

1. INTRODUÇÃO

Este relatório se refere ao projeto de um reprodutor de músicas desenvolvido com a placa Arduino UNO, proposto como trabalho da disciplina de Digital Experience Ultimate, elaborado em duplas.

O sistema projetado permite que o usuário escolha uma música em um menu exibido em um display LCD, possibilitando selecionar, reproduzir, pausar, retomar e interromper cinco melodias armazenadas na memória flash do microcontrolador. A lógica de controle foi implementada como uma máquina de estados finitos. O som é produzido por um buzzer, e dois LEDs indicam se o equipamento está tocando ou pausado.

O projeto foi desenvolvido e simulado na plataforma Tinkercad e preparado para execução em hardware físico por meio de compilação condicional. As soluções adotadas asseguraram resposta imediata aos comandos do usuário e uso eficiente da memória RAM disponível.

2. OBJETIVOS

2.1 Objetivo Geral

a) Implementar um menu de músicas navegável, exibido no display LCD;

b) Reproduzir melodias no buzzer, respeitando a frequência e a duração de cada nota;

c) Disponibilizar as funções de reprodução, pausa, retomada e interrupção;

d) Exibir ao usuário o nome da música em execução e o percentual já reproduzido;

e) Sinalizar o estado de operação do equipamento por meio de LEDs;

f) Oferecer modos de teste para verificação da montagem física do circuito.

3.MATERIAIS

3.1 Componentes de Hardware

- Uma Placa Arduino UNO – Unidade de processamento e controle;
- Um Display LCD 16x2 com módulo I2C – Exibição do menu e do estado de reprodução;

- Um Buzzer piezoelétrico – Geração do sinal sonoro;
- Quatro botões (push button) - Comandos UP, DOWN, PLAY/PAUSE e STOP;
- Um LED verde - Indicação de reprodução em andamento;
- Um LED vermelho - Indicação de reprodução em pausa;
- Dois resistores de 220 Ω – Limitação de corrente dos LEDs;
- Um resistor de 110 Ω – Limitação de corrente do buzzer;
- Protoboard – Base para a montagem do circuito;
- Fios de ligação (jumpers) – Interligação dos componentes;

3.2 Componentes de Software

Tinkercad (Autodesk): ambiente de simulação, no qual se utiliza a biblioteca Adafruit_LiquidCrystal;

4. MONTAGEM DO CIRCUITO

A montagem foi realizada inicialmente no ambiente Tinkercad. A Figura 1 apresenta a disposição física dos componentes na protoboard, a Figura 2 apresenta o diagrama esquemático gerado pela própria plataforma e o quadro sintetiza as conexões entre os componentes e os pinos do Arduino

Componente	**Terminal**	**Pino do Arduino**
Display LCD (módulo I2C)	SDA	A4
Display LCD (módulo I2C)	SCL	A5
Display LCD (módulo I2C)	VCC / GND	5V / GND
Botão UP (sobe no menu)	Terminal de sinal	A0
Botão DOWN (desce no menu)	Terminal de sinal	A1
Botão PLAY/PAUSE	Terminal de sinal	A2
Botão STOP	Terminal de sinal	A3
LED verde (reprodução), com R2 de 220 Ω	Ânodo	D9
LED vermelho (pausa), com R1 de 220 Ω	Ânodo	D8
Buzzer, com R5 de 100 Ω em série	Positivo (+)	D6
Botões, LEDs e buzzer	Terminais de retorno	GND

Figura 1 – Montagem do circuito no ambiente Tinkercad

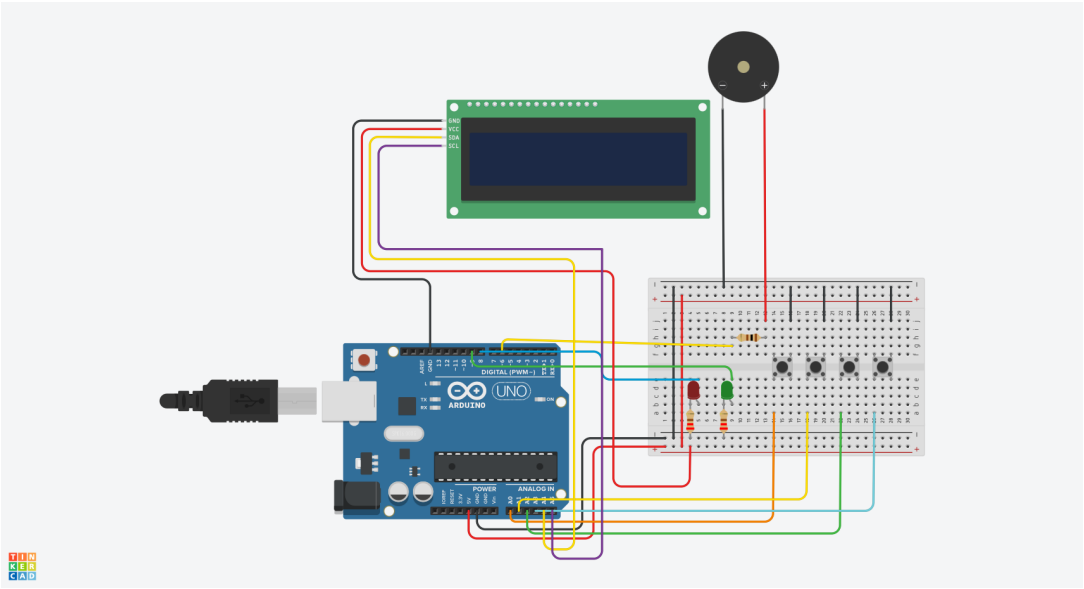
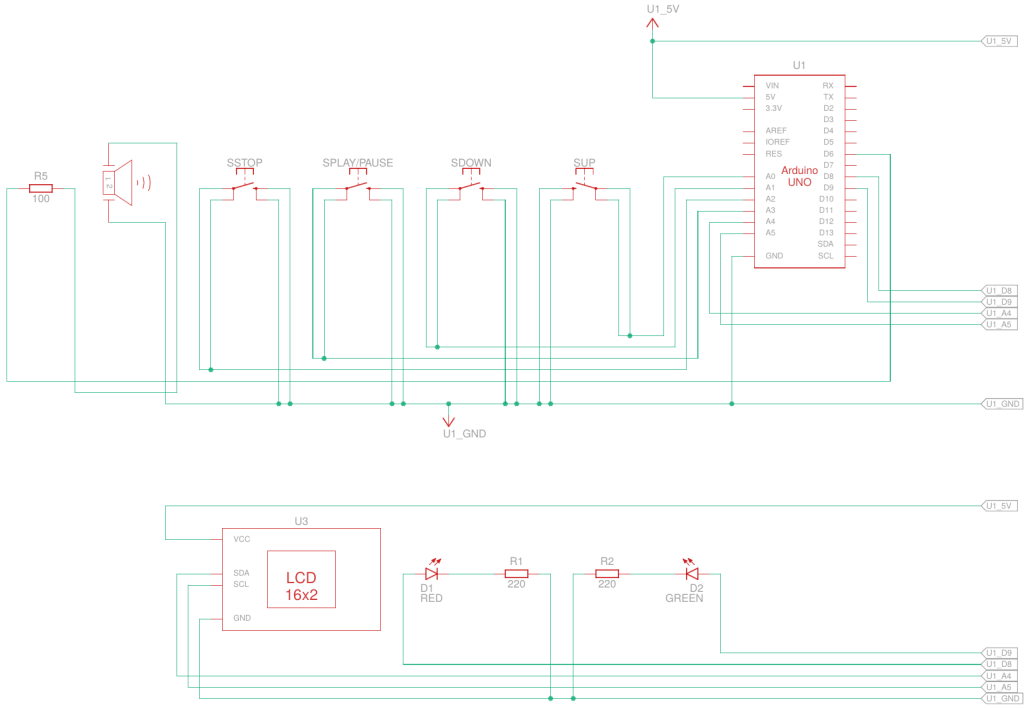


Figura 2 – Diagrama esquemático do circuito



Os botões foram configurados no modo INPUT_PULLUP, que ativa o resistor de pull-up interno do microcontrolador associado ao pino (ARDUINO, 2026b). Assim, com o botão em repouso, o pino permanece em nível lógico alto (HIGH). Cada botão possui um terminal ligado ao respectivo pino e o outro ligado ao GND; quando acionado, conecta o pino ao terra e a leitura passa a nível lógico baixo (LOW). Adota-se, portanto, lógica invertida, na qual o valor LOW indica

botão pressionado. Essa configuração dispensa resistores externos para os botões e reduz o número de componentes do circuito.

Cada LED foi ligado em série com um resistor de 220 Ω, e o buzzer, em série com um resistor de 100 Ω, de modo a limitar a corrente fornecida pelos pinos de saída. O buzzer foi conectado ao pino digital 6. A função tone(), responsável pela geração do som, produz uma onda quadrada com ciclo de trabalho de 50% na frequência especificada e pode ser aplicada a qualquer pino digital; contudo, seu uso interfere na saída PWM dos pinos 3 e 11 nas placas diferentes da Arduino Mega (ARDUINO, 2026d). Essa restrição não afeta o presente projeto, pois tais pinos não são utilizados.

5.FUNCIONAMENTO GERAL

Ao ser energizado, o sistema exibe durante um segundo a mensagem de inicialização “Reprodutor / Arduino UNO” e, em seguida, apresenta o menu com as músicas disponíveis, no qual a opção selecionada é indicada por uma seta no início da linha. Foram cadastradas cinco músicas:

a) Harry Potter (Hedwig’s Theme);
b) Rei Leão (The Lion Sleeps Tonight);
c) Star Wars (tema principal);
d) Take On Me (a-ha);
e) Tubarão.

A operação do sistema por meio dos botões ocorre da seguinte forma:

a) No menu, o botão UP seleciona a música anterior e o botão DOWN seleciona a seguinte, com navegação circular: após a última música, o menu retorna à primeira;
b) No menu, o botão PLAY/PAUSE inicia a reprodução da música selecionada;
c) Durante a reprodução, o botão PLAY/PAUSE suspende a execução e o botão STOP a encerra, retornando ao menu;
d) Durante a pausa, o botão PLAY/PAUSE retoma a execução e o botão STOP retorna ao menu;
e) Ao término da música, o sistema retorna automaticamente ao menu.

Durante a reprodução, a primeira linha do display apresenta a indicação “Reproduzindo” (ou “Pausado”) acompanhada do percentual concluído, e a segunda linha apresenta o nome da música. O LED verde permanece aceso

enquanto a música é executada e o LED vermelho, enquanto ela está pausada; no menu, ambos permanecem apagados.

6.DESENVOLVIMENTO DO PROGRAMA

6.1 Configurações Iniciais

O início do código define duas constantes de configuração. A primeira, TINKERCAD, seleciona o ambiente de execução: o valor 1 corresponde ao simulador e o valor 0, à placa física. Essa distinção é necessária porque as bibliotecas de controle do display diferem entre os ambientes; por isso, o código emprega compilação condicional para incluir apenas a biblioteca adequada. A segunda constante, MODO_TESTE, seleciona o modo de operação e é detalhada na seção 7.

Na placa física, o endereço I2C do display foi definido como 0x27. Como alguns módulos utilizam o endereço 0x3F, recomenda-se alterar esse valor caso o display não seja inicializado.

6.2 Representação das Notas Musicais

As frequências das notas e os andamentos das músicas foram baseados no repositório arduino-songs (COUTO, 2019) e adaptados ao uso de PROGMEM e aos requisitos deste projeto. Cada nota é armazenada em uma estrutura denominada Nota, composta por dois campos: a frequência, em hertz, e o divisor da nota inteira.

A frequência determina a altura do som; por exemplo, a nota Lá da quarta oitava (NOTE_A4) corresponde a 440 Hz. As pausas são representadas pela frequência zero (REST). O divisor determina a duração relativa da nota: o valor 4 representa uma semínima (um quarto da nota inteira), o valor 8, uma colcheia (um oitavo) e o valor 16, uma semicolcheia. Divisores negativos indicam notas pontuadas, cuja duração corresponde a uma vez e meia a duração normal. São exemplos de notas definidas no programa:

a) Semínima de Sol: nota Sol com divisor 4;
b) Semínima pontuada de Sol: nota Sol com divisor −4;
c) Pausa de mínima: frequência zero com divisor 2.

6.3 Armazenamento das Melodias

O microcontrolador ATmega328P do Arduino UNO dispõe de apenas 2 KB de memória SRAM, ao passo que sua memória de programa (flash) possui 32 KB, dos quais 0,5 KB são ocupados pelo bootloader (ARDUINO, 2026e). Como as melodias são extensas, mantê-las na SRAM comprometeria o espaço necessário à execução do programa. Por esse motivo, os vetores de notas foram

declarados com o modificador PROGMEM, que instrui o compilador a manter os dados na memória flash em vez de copiá-los para a SRAM na inicialização (ARDUINO, 2026c). A leitura de cada nota é realizada pelas funções pgm_read_word, para a frequência (dois bytes), e pgm_read_byte, para o divisor (um byte).

Cada nota ocupa, portanto, 3 bytes. A música Tubarão, composta por 40 notas, ocupa aproximadamente 120 bytes da memória flash. Para evitar a repetição de trechos idênticos no código-fonte, foram utilizadas macros, como TUB_BLOCO, TAKE_BLOCO e as macros da música Rei Leão. Cada macro agrupa um trecho recorrente, e a melodia é composta pela inserção desse trecho quantas vezes forem necessárias.

6.4 Cadastro das Músicas

As músicas são organizadas em um vetor de estruturas denominadas Musica. Cada estrutura contém o nome da música (com até 14 caracteres, para caber no display juntamente com a seta de seleção), o vetor de notas, a quantidade de notas, o andamento em batidas por minuto (BPM) e o percentual de aceleração ao longo da execução. O número total de músicas é calculado automaticamente pelo programa, de modo que a inclusão de uma nova música requer apenas o acréscimo de uma linha no vetor.

6.5 Leitura dos Botões e Debounce

Botões mecânicos apresentam o fenômeno de trepidação (bounce): ao serem pressionados, seus contatos abrem e fecham diversas vezes em poucos milissegundos antes de se estabilizarem, o que poderia ser interpretado como múltiplos acionamentos. Para mitigar esse efeito, o programa aceita uma mudança de leitura somente se tiverem decorrido mais de 40 ms desde a última mudança aceita, técnica denominada debounce por software.

A função lerBotoes percorre os quatro botões e, ao detectar a transição de solto para pressionado, registra um clique válido. Esse registro permanece ativo por apenas uma iteração do laço principal, de modo que cada acionamento gera uma única ação. Os eventos também são enviados ao Monitor Serial, a uma taxa de 9600 bauds, o que auxilia a depuração.

6.6 Máquina de Estados

O programa é controlado por uma máquina de estados finitos com três estados: MENU, no qual o usuário navega pelas músicas e os LEDs permanecem apagados; TOCANDO, no qual a música está em execução e o LED verde permanece aceso; e PAUSADO, no qual a execução está suspensa e o LED

vermelho permanece aceso. O Quadro descreve as transições entre esses estados.

Estado atual	Evento	Próximo estado	Ação associada
MENU	UP ou DOWN	MENU	Altera a música selecionada
MENU	PLAY/PAUSE	TOCANDO	Inicia a reprodução; acende o LED verde
TOCANDO	PLAY/PAUSE	PAUSADO	Suspende a reprodução; acende o LED vermelho
TOCANDO	STOP	MENU	Encerra a reprodução; apaga os LEDs
TOCANDO	Fim da música	MENU	Retorna ao menu; apaga os LEDs
PAUSADO	PLAY/PAUSE	TOCANDO	Retoma a nota atual desde o início
PAUSADO	STOP	MENU	Retorna ao menu; apaga os LEDs

Um aspecto central da implementação é a ausência da função delay() durante a reprodução. Em seu lugar, o programa registra, por meio da função millis(), o instante de início de cada nota e verifica, a cada iteração do laço principal, se o tempo previsto para a nota já se esgotou; em caso afirmativo, avança para a nota seguinte. Dessa forma, o sistema continua a ler os botões ininterruptamente e responde de imediato aos comandos do usuário, mesmo durante a execução de uma nota. Caso a função delay() fosse utilizada, os botões só seriam lidos ao final de cada nota, e o equipamento aparentaria estar travado.

Ao retomar uma música pausada, o programa reinicia a nota atual desde o seu início, e não a partir do ponto exato em que a execução foi interrompida.

6.7 Controle do Display

A função linha escreve um texto em uma das duas linhas do display e o completa com espaços até 16 caracteres. Esse procedimento sobrescreve qualquer conteúdo anterior da linha e dispensa a limpeza completa da tela, operação que provocaria cintilação.

O menu exibe duas músicas por vez. A variável topoMenu armazena o índice da música apresentada na primeira linha e é ajustada pelo programa de modo que a música selecionada permaneça sempre visível, o que viabiliza a rolagem de cinco músicas em apenas duas linhas.

O percentual de reprodução é obtido pelo produto entre o índice da nota atual e 100, dividido pelo total de notas; por exemplo, na nota 10 de uma música com

40 notas, o valor é 25%. Para reduzir o tráfego no barramento I2C, o percentual somente é reescrito no display quando seu valor se altera.

7. MODOS DE TESTE

Considerando a frequência de erros de ligação em montagens físicas, o programa dispõe de dois modos de teste, selecionados pela constante MODO_TESTE, conforme o quadro.

Valor	Modo	Comportamento
0	Operação normal	Funcionamento regular do reprodutor.
1	Teste dos botões	O display exibe “U0 D0 P0 S0”, em que cada letra corresponde a um botão (UP, DOWN, PLAY e STOP) e o dígito 1 indica botão pressionado. Adicionalmente, UP acende o LED verde, DOWN acende o LED vermelho, PLAY emite um tom de 440 Hz e STOP, um tom de 880 Hz, o que permite verificar todos os botões, os LEDs e o buzzer.
2	Descoberta de pinos	O programa percorre os pinos 6, 7, 8, 9, 10 e 13. Em cada um, aciona o pino por 1,2 s (para identificar um LED) e, em seguida, emite um tom por 1,2 s (para identificar o buzzer), exibindo no display o pino em teste. Esse modo é útil quando há dúvida sobre os pinos aos quais o buzzer e os LEDs foram conectados.

8. PROCEDIMENTOS DE EXECUÇÃO

8.1 Simulação no Tinkercad

a) Montar o circuito conforme a seção 4;
b) Inserir o código no editor da plataforma, mantendo a constante TINKERCAD com valor 1;
c) Iniciar a simulação.

8.2 Execução em Placa de Arduino UNO Física

a) Alterar o valor da constante TINKERCAD para 0;
b) Instalar a biblioteca LiquidCrystal_I2C por meio do Gerenciador de Bibliotecas da Arduino IDE;
c) Selecionar a placa Arduino UNO e a porta de comunicação correspondente;
d) Compilar e transferir o programa para a placa;
e) Caso o display não seja inicializado, testar o endereço 0x3F e ajustar o contraste no potenciômetro do módulo I2C.

9. INCLUSÃO DE NOVAS MÚSICAS

A estrutura do programa permite ampliar o repertório com poucas alterações, observando-se as seguintes etapas:

a) Criar um novo vetor de notas, declarado com PROGMEM, no mesmo formato dos vetores existentes;
b) Caso a música utilize uma nota cuja frequência ainda não esteja definida, acrescentar a definição correspondente à lista de frequências;
c) Adicionar uma linha ao vetor de músicas, informando o nome (com até 14 caracteres), o vetor de notas, a quantidade de notas, o andamento em BPM e o percentual de aceleração (zero para músicas sem aceleração).

Como o total de músicas é calculado automaticamente, nenhuma outra modificação no código é necessária.

10. CONCLUSÃO

O projeto atingiu os objetivos propostos. O reprodutor executa as cinco músicas cadastradas, responde de imediato aos comandos dos botões, informa o estado de operação por meio do display e dos LEDs e pode ser utilizado tanto no simulador quanto em uma placa Arduino física.

Entre as decisões técnicas que asseguraram o funcionamento adequado do sistema, destacam-se: o armazenamento das melodias na memória flash, que preservou a memória RAM; a máquina de estados associada à temporização por millis(), que eliminou o uso de delay() e manteve os botões permanentemente responsivos; e o debounce por software, que evitou leituras espúrias. Os modos de teste mostraram-se úteis para a identificação rápida de falhas de ligação.

Como trabalhos futuros, propõem-se: a retomada da música a partir do ponto exato da pausa; a implementação de controle de volume; a ampliação do repertório; e o armazenamento das melodias em cartão de memória, o que eliminaria a limitação imposta pela memória interna do microcontrolador.

11. REFERÊNCIAS

ARDUINO. **Arduino Documentation**. [*S. l.*]: Arduino, 2026a. Disponível em: https://docs.arduino.cc/. Acesso em: 27 out. 2026.

ARDUINO. pinMode(). *In*: ARDUINO. **Arduino Language Reference**. [*S. l.*]: Arduino, 2026b. Disponível em: https://docs.arduino.cc/language-reference/en/functions/digital-io/pinMode/. Acesso em: 27 out. 2026.

ARDUINO. PROGMEM. *In*: ARDUINO. **Arduino Language Reference**. [*S. l.*]: Arduino, 2026c. Disponível em: https://docs.arduino.cc/language-reference/en/variables/utilities/PROGMEM/. Acesso em: 29 out. 2026.

ARDUINO. tone(). *In*: ARDUINO. **Arduino Language Reference**. [*S. l.*]: Arduino, 2026d. Disponível em: https://docs.arduino.cc/language-reference/en/functions/advanced-io/tone/. Acesso em: 29 out. 2026.

ARDUINO. **UNO R3**. [*S. l.*]: Arduino, 2026e. Disponível em: https://docs.arduino.cc/hardware/uno-rev3/. Acesso em: 1 nov. 2026.

AUTODESK. **Tinkercad**. [*S. l.*]: Autodesk, 2026. Disponível em: https://www.tinkercad.com/. Acesso em: 1 nov. 2026.

COUTO, Robson. **arduino-songs**. [*S. l.*]: GitHub, 2019. Disponível em: https://github.com/robsoncouto/arduino-songs. Acesso em: 1 nov. 2026.